LAPORAN UJIAN TENGAH SEMESTER

PostgreSQL dan PGAdmin via Docker Compose, Skema SALAM, Tiga Role User, dan Bukti Akses Internal-Eksternal

Mata Kuliah Manajemen Basis Data (20100470501024), Kelas V/A-D, Tahun Akademik 2026/2027

Nama	M. Ariel Davinsyah Putra
NIM	1247050122
Mata Kuliah	Manajemen Basis Data (kode 20100470501024)
Kelas	V/A-D
Dosen Pengampu	Wildan Budiawan Z, S.T., M.Kom.
Tahun Akademik	2026/2027, Semester Ganjil
Tanggal	1 November 2026

Semua tangkapan layar dan transkrip pada laporan ini diambil langsung dari komputer yang digunakan saat mengerjakan tugas. Versi perangkat yang dipakai: GIT 2.51.0, Docker Desktop 4.71.0 (Engine 29.4.1), PostgreSQL 15.19 (image postgres:15-alpine, memenuhi syarat versi 9 s.d. 15), psql client 18.3, pgAdmin 9.18 (container dpage/pgadmin4:9.18), DBeaver Community 26.2.0, dan VS Code 1.99.

Ringkasan konfigurasi (seluruh nilai dibaca dari berkas .env, tidak ada yang di-hardcode di docker-compose.yml):

Layanan	Container	Alamat	User	Password
PostgreSQL 15.19	postgres_ariel	host: localhost:22122, dalam docker: postgres_ariel:5432	postgres	ifunggul (di .env)
pgAdmin 9	pgadmin_ariel	http://localhost:44122	ariel.davinsyah@yopmail.com	arielpg122 (di .env)

1. Container PostgreSQL: postgres_ariel (bobot 30 poin)

Bagian ini membuktikan tiga tuntutan soal: container bernama postgres_ariel memakai PostgreSQL versi 15 (rentang sah 9 s.d. 15), user postgres dengan password ifunggul yang disembunyikan di berkas .env (tidak di-hardcode), dan port 22XXX sesuai tiga digit terakhir NIM. NIM 1247050122 berarti tiga digit terakhir 122, sehingga port PostgreSQL adalah 22 + 122 = 22122.

1.1 Struktur proyek

```
uts-manajemen-basis-data/
|-- docker-compose.yml     # 2 container, nol kredensial hardcode
|-- .env                   # kredensial + port (sengaja di-commit demi penilaian)
|-- .env.example           # templat .env
|-- README.md              # dokumentasi lengkap
|-- db/
|   |-- init/              # auto-init saat boot pertama container
|   |   |-- 01_schema.sql  # skema SALAM + tabel + view + trigger + data
|   |   `-- 02_roles.sql   # tiga role user + grant
|   `-- uji/               # skrip pengujian bukti (dijalankan manual)
|       |-- uji_constraint.sql      |-- uji_backend_dev.sql
|       |-- uji_bi_dev.sql          |-- uji_data_engineer.sql
|       `-- dbeaver_eksternal.sql
|-- bukti/                # 10 transkrip output asli pengujian
`-- laporan/              # laporan ini (DOCX + PDF)
```

1.2 Isi berkas .env

Password tidak pernah ditulis di docker-compose.yml. Compose membaca nilai dari .env, dan setiap variabel ditulis dengan bentuk ${VAR:?pesan} sehingga jika .env belum ada, compose langsung gagal dengan pesan yang menyebut variabel yang kurang. Perilaku ini dibuktikan nyata pada transkrip 1.4 (simulasi .env disembunyikan).

```
# Konfigurasi UTS Manajemen Basis Data
# Port mengikuti 3 digit terakhir NIM (122):
#   PostgreSQL 22XXX -> 22122 , PGAdmin 44XXX -> 44122
POSTGRES_USER=postgres
POSTGRES_PASSWORD=ifunggul
POSTGRES_DB=uts_mbd
POSTGRES_PORT=22122
PGADMIN_DEFAULT_EMAIL=ariel.davinsyah@yopmail.com
PGADMIN_DEFAULT_PASSWORD=arielpg122
PGADMIN_PORT=44122
```

Berkas .env sengaja di-commit karena soal meminta dokumentasi isinya untuk keperluan penilaian; peringatan untuk konteks produksi ditulis di README.

1.3 Isi docker-compose.yml

```
services:
  postgres:
    image: postgres:15-alpine
    container_name: postgres_ariel
    environment:
```

```
      POSTGRES_USER: ${POSTGRES_USER:?Setel POSTGRES_USER di file .env}
      POSTGRES_PASSWORD: ${POSTGRES_PASSWORD:?Setel POSTGRES_PASSWORD di file .env}
      POSTGRES_DB: ${POSTGRES_DB:?Setel POSTGRES_DB di file .env}
      PGDATA: /data/postgres
    ports:
      # 127.0.0.1: port hanya terbuka di komputer lokal (hasil audit)
      - "127.0.0.1:${POSTGRES_PORT:?Setel POSTGRES_PORT di file .env}:5432"
    volumes:
      - pgdata_ariel:/data/postgres
      - ./db/init:/docker-entrypoint-initdb.d:ro
    healthcheck:
      test: ["CMD-SHELL", "pg_isready -U $$POSTGRES_USER -d $$POSTGRES_DB"]
      interval: 5s
      timeout: 5s
      retries: 10
    restart: unless-stopped
    networks:
      - net_ariel

  pgadmin:
    image: dpage/pgadmin4:9.18   # versi dikunci
    container_name: pgadmin_ariel
    environment:
      PGADMIN_DEFAULT_EMAIL: ${PGADMIN_DEFAULT_EMAIL:?Setel PGADMIN_DEFAULT_EMAIL di .env}
      PGADMIN_DEFAULT_PASSWORD: ${PGADMIN_DEFAULT_PASSWORD:?Setel PGADMIN_DEFAULT_PASSWORD
di .env}
      PGADMIN_CONFIG_SERVER_MODE: "False"
      PGADMIN_CONFIG_MASTER_PASSWORD_REQUIRED: "False"
    ports:
      # 127.0.0.1: pgAdmin mode desktop tanpa login, wajib lokal saja
      - "127.0.0.1:${PGADMIN_PORT:?Setel PGADMIN_PORT di file .env}:80"
    volumes:
      - pgadmin_ariel:/var/lib/pgadmin
    depends_on:
      postgres:
        condition: service_healthy
    restart: unless-stopped
    networks:
      - net_ariel

volumes:
  pgdata_ariel:
  pgadmin_ariel:

networks:
  net_ariel:
    driver: bridge
```

1.4 Bukti menjalankan (transkrip asli, lengkap di bukti/01)

```
$ docker compose up -d
```

```
 Container postgres_ariel Healthy
 Container pgadmin_ariel Started

$ docker ps --filter name=ariel
NAMES            IMAGE                STATUS                    PORTS
pgadmin_ariel    dpage/pgadmin4       Up 14 seconds             0.0.0.0:44122->80/tcp
postgres_ariel   postgres:15-alpine   Up 20 seconds (healthy)   0.0.0.0:22122->5432/tcp

=== BUKTI .env WAJIB: simulasi tanpa .env ===
$ mv .env /tmp/env-disembunyikan && docker compose up -d
error while interpolating services.postgres.environment.POSTGRES_USER:
required variable POSTGRES_USER is missing a value:
Setel POSTGRES_USER di file .env

=== LOG BOOT: skrip init dijalankan otomatis ===
$ docker logs postgres_ariel 2>&1 | grep -E 'initdb.d|ready to accept'
/usr/local/bin/docker-entrypoint.sh: running /docker-entrypoint-initdb.d/01_schema.sql
/usr/local/bin/docker-entrypoint.sh: running /docker-entrypoint-initdb.d/02_roles.sql
PostgreSQL init process complete; ready for start up.
2026-11-01 09:32:27.861 UTC [1] LOG:  database system is ready to accept connections
```

Versi server yang berjalan dibuktikan dari luar container di Bab 2 (psql host membaca: PostgreSQL 15.19 on x86_64-pc-linux-musl).

Hardening setelah audit (bukti/10-hardening-port.txt)

Pengerjaan diaudit keamanannya sebelum dipublikasikan. Hasil audit menemukan port tadinya terikat ke seluruh antarmuka (0.0.0.0) padahal pgAdmin mode desktop terbuka tanpa form login, sehingga satu-satunya perbaikan yang dibutuhkan adalah mengikat kedua port ke 127.0.0.1 dan mengunci versi image pgAdmin. Setelah diterapkan, container dibuat ulang dan semuanya diverifikasi ulang:

```
$ docker compose up -d        # recreate karena compose berubah
NAMES            STATUS                    PORTS
pgadmin_ariel    Up 30 seconds             127.0.0.1:44122->80/tcp
postgres_ariel   Up 36 seconds (healthy)   127.0.0.1:22122->5432/tcp

=== netstat: hanya mendengarkan loopback ===
TCP    127.0.0.1:22122    0.0.0.0:0    LISTENING    15676
TCP    127.0.0.1:44122    0.0.0.0:0    LISTENING    15676

=== akses eksternal dari host tetap jalan (bi_dev via localhost:22122) ===
 1 | 1247050122 | M. Ariel Davinsyah Putra
 2 | 1247050101 | Bunga Aulia Rahma
 3 | 1247050110 | Damar Wicaksana
(3 rows)
```

1.5 Fitur pembeda pada container

- **FITUR PEMBEDA: healthcheck pg_isready + depends_on condition service_healthy.** pgadmin_ariel tidak dinyalakan sebelum postgres_ariel benar-benar siap menerima koneksi, bukan sekadar prosesnya hidup; status (healthy) tampak pada docker ps.
- **FITUR PEMBEDA: Interpolasi wajib ${VAR:?pesan}.** Tanpa .env, compose menolak jalan dengan pesan jelas (dibuktikan di 1.4), bukan diam-diam memakai nilai default.
- **FITUR PEMBEDA: Volume dan network bernama (pgdata_ariel, pgadmin_ariel, net_ariel).** Data bertahan antar restart, keduanya container satu jaringan bridge sendiri sehingga akses internal tidak keluar docker.
- **FITUR PEMBEDA: Folder ./db/init ter-mount baca-saja ke /docker-entrypoint-initdb.d.** Skema, role, dan data awal tercipta otomatis pada boot pertama, terbukti pada log 1.4.
- **FITUR PEMBEDA: Skrip init idempoten untuk penilaian.** Setiap skrip uji membersihkan jejaknya sendiri sehingga dapat dijalankan ulang dosen dengan hasil sama.
- **FITUR PEMBEDA: Pengikat port 127.0.0.1 dan versi image dikunci (dpage/pgadmin4:9.18), hasil audit keamanan pengerjaan sendiri.** Kedua port hanya mendengarkan komputer lokal sehingga pgAdmin tanpa login tidak pernah terekspos ke jaringan kampus, sementara akses eksternal dari host tetap berjalan (bukti/10).

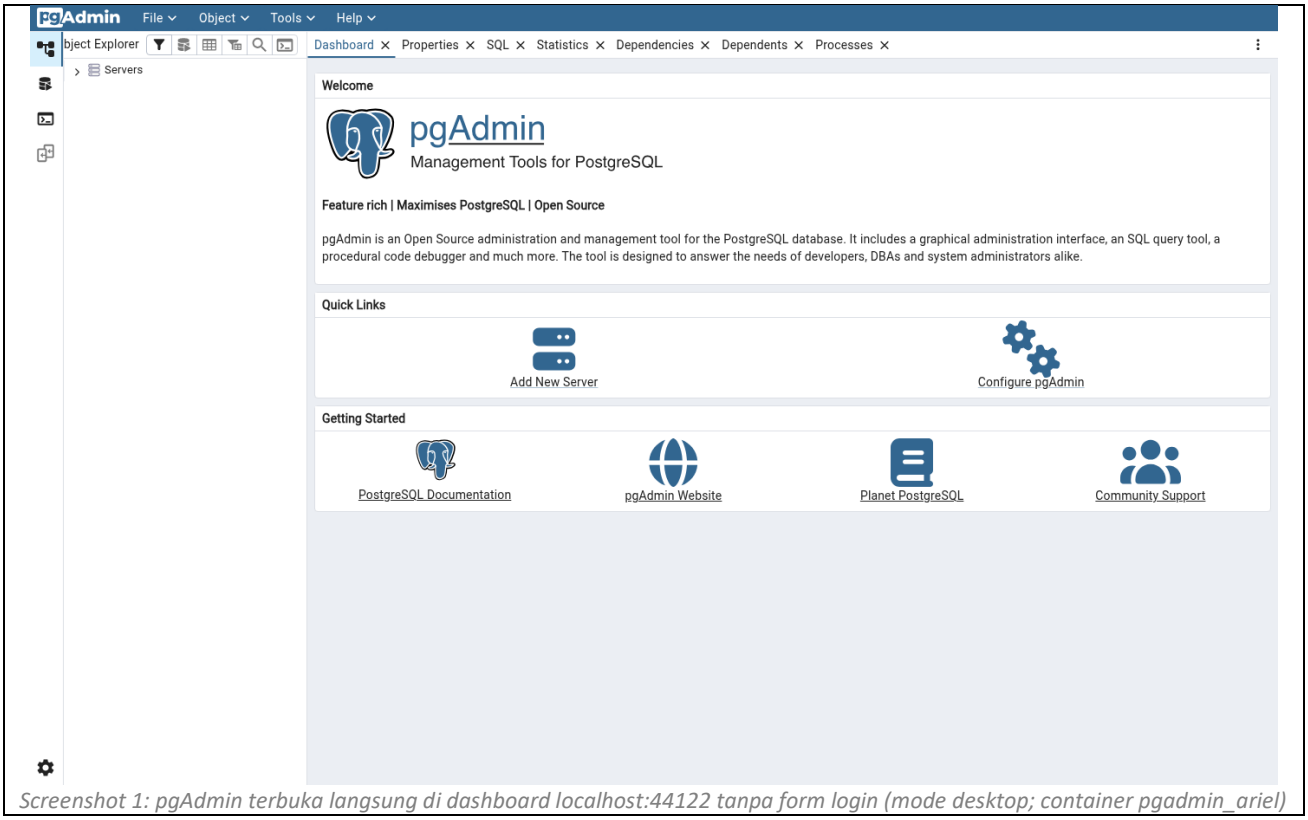
2. Container PGAdmin dan Bukti Akses Internal-Eksternal (bobot 20 poin)

Soal meminta container pgadmin_ariel di port 44XXX (44122) serta bukti PostgreSQL dapat diakses dari internal docker (lewat PGAdmin4) dan eksternal docker (lewat DBeaver). Kedua arah dibuktikan berbeda caranya karena jalurnya memang berbeda: internal memakai nama container di jaringan docker, eksternal memakai port mapping di komputer host.

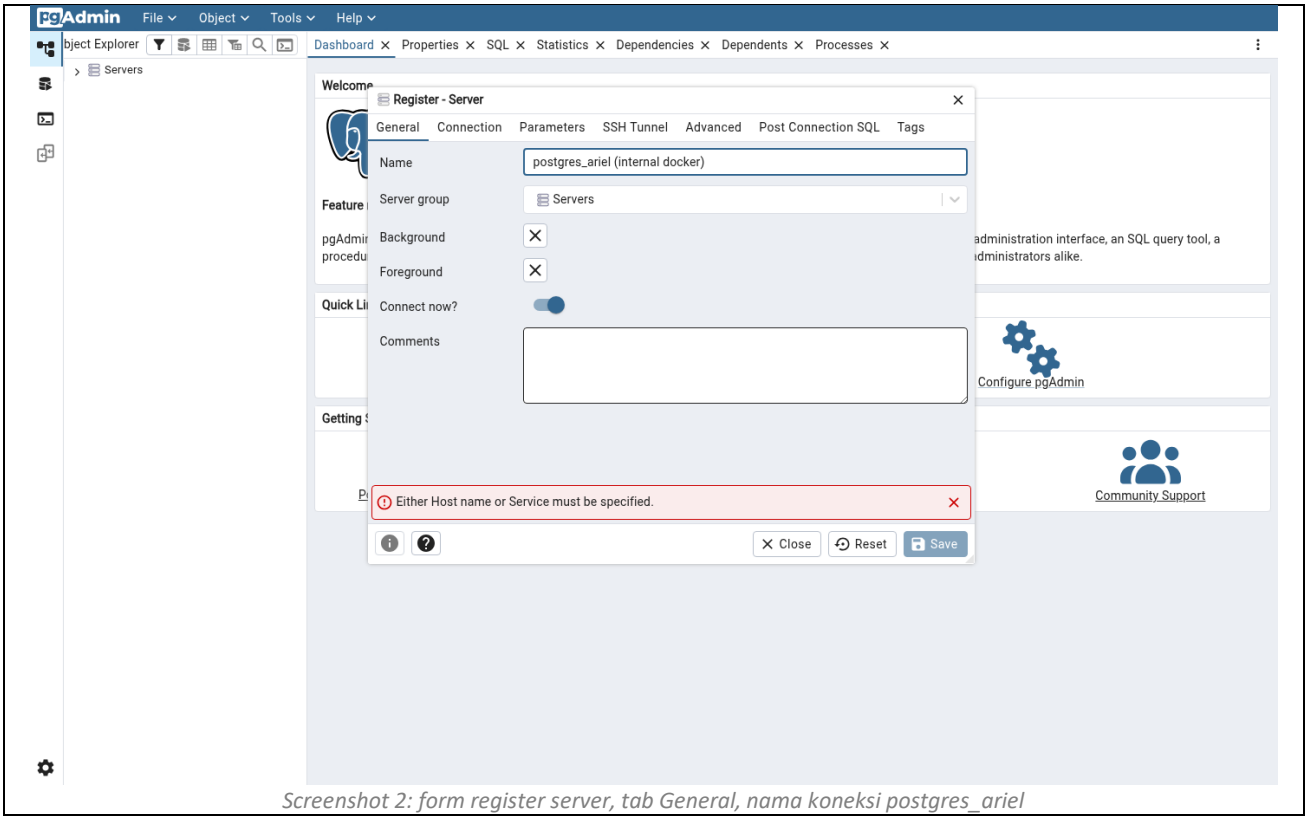
Arah	Alamat yang dipakai client	Alat bukti	Bukti
Internal docker	postgres_ariel:5432 (nama container, jaringan net_ariel)	pgAdmin 9 di browser	Screenshot 1-7, transkrip tidak perlu keluar docker
Eksternal docker	localhost:22122 (port mapping host)	psql 18.3 dan DBeaver Community 26.2.0 di host	bukti/07 (psql, tiga role), bukti/08 (DBeaver + netstat)

2.1 Akses internal: pgAdmin -> postgres_ariel

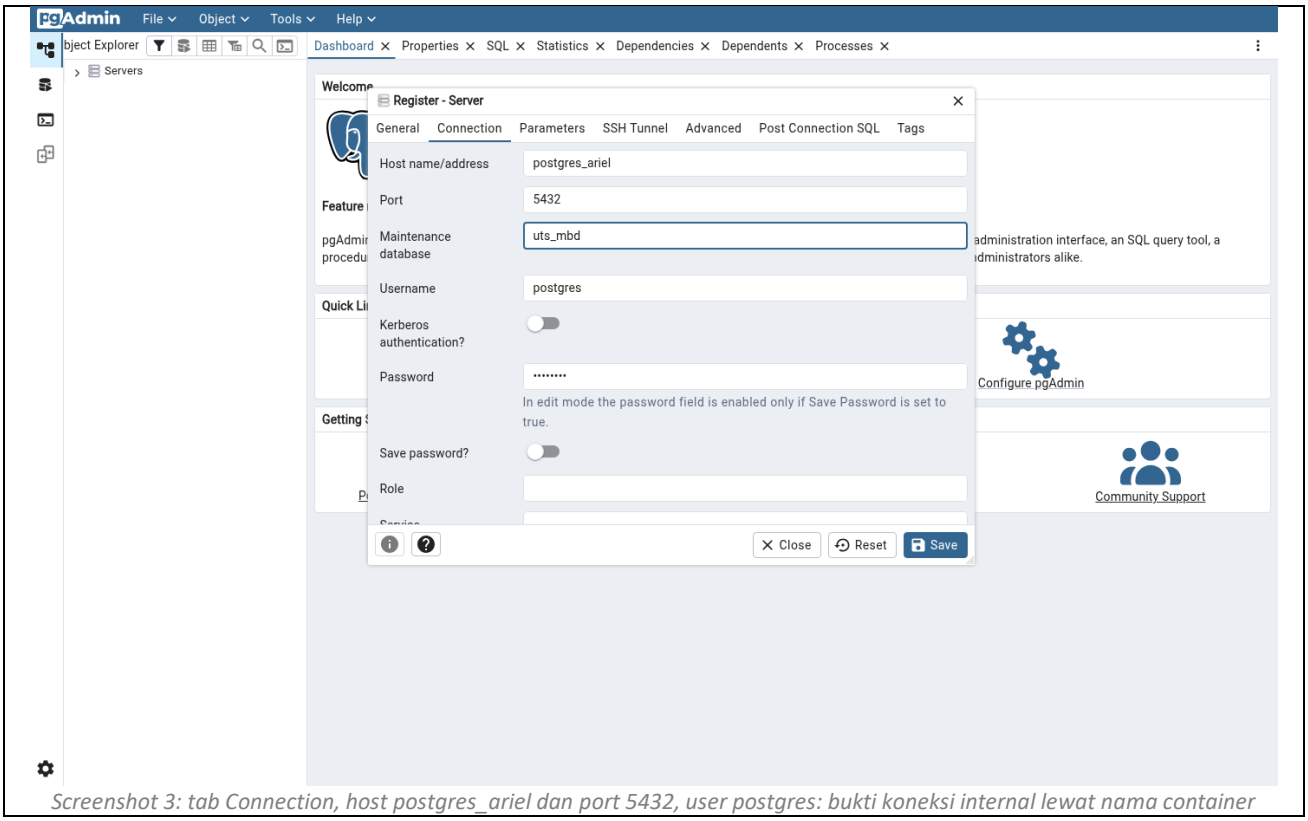
pgAdmin dibuka di http://localhost:44122. Karena dikonfigurasi mode desktop (SERVER_MODE False), dashboard terbuka langsung tanpa form login; kedua variabel PGADMIN_DEFAULT_* tetap disetel di .env karena compose mewajibkannya ada, dan port-nya diikat ke 127.0.0.1. Server kemudian didaftarkan dengan host postgres_ariel (nama container, bukan localhost) dan port 5432. Koneksi ini berjalan sepenuhnya di dalam jaringan docker net_ariel.



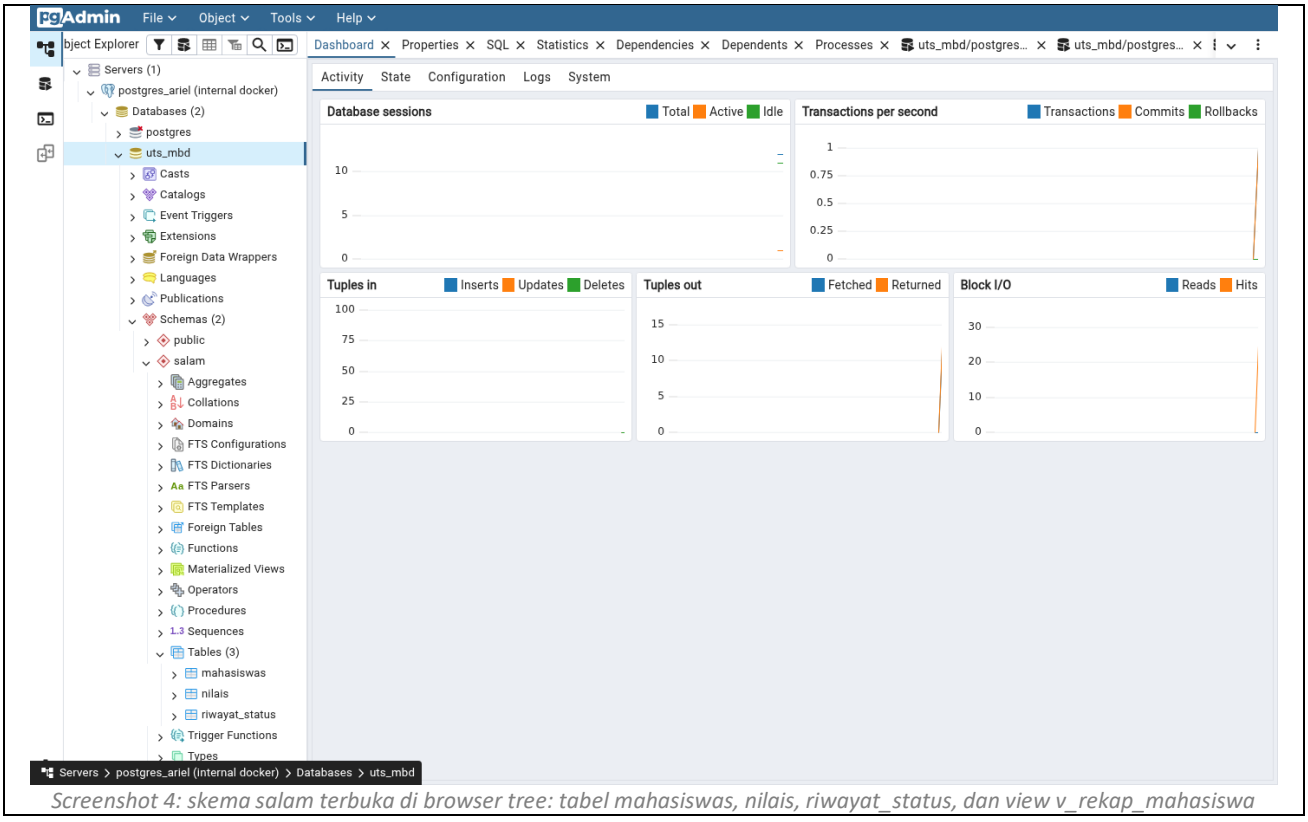
Screenshot 1: pgAdmin terbuka langsung di dashboard localhost:44122 tanpa form login (mode desktop; container pgadmin_ariel)



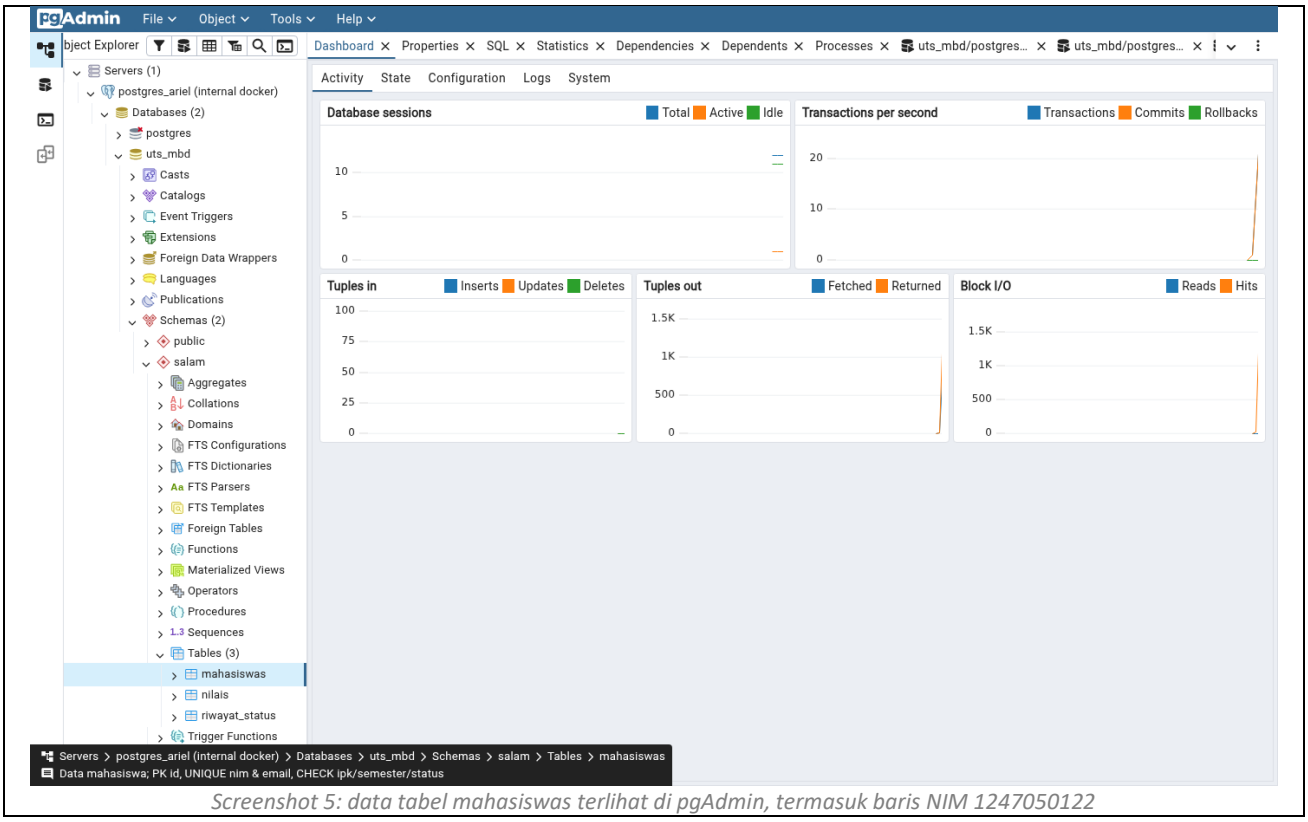
Screenshot 2: form register server, tab General, nama koneksi postgres_ariel



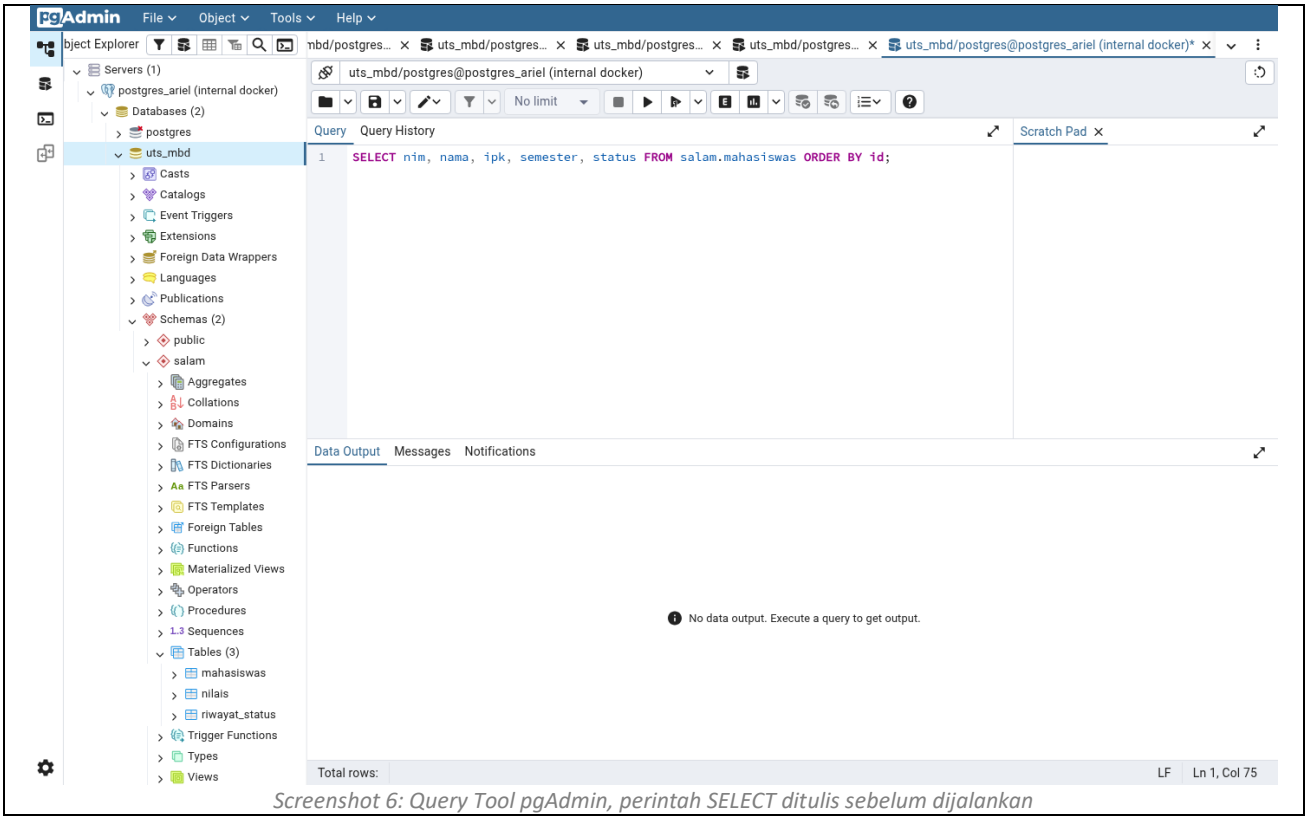
Screenshot 3: tab Connection, host postgres_ariel dan port 5432, user postgres: bukti koneksi internal lewat nama container



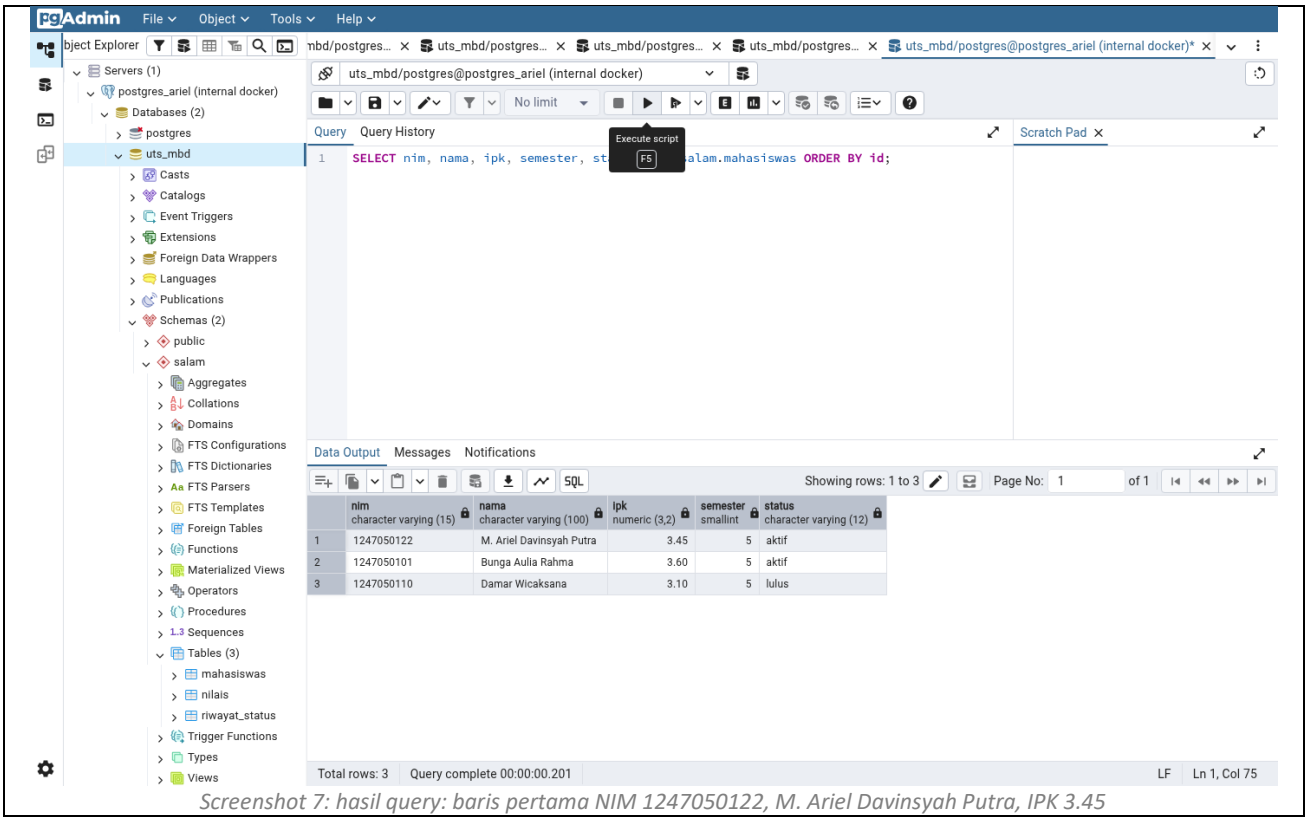
Screenshot 4: skema salam terbuka di browser tree: tabel mahasiswas, nilais, riwayat_status, dan view v_rekap_mahasiswa



Screenshot 5: data tabel mahasiswas terlihat di pgAdmin, termasuk baris NIM 1247050122



Screenshot 6: Query Tool pgAdmin, perintah SELECT ditulis sebelum dijalankan



Screenshot 7: hasil query: baris pertama NIM 1247050122, M. Ariel Davinsyah Putra, IPK 3.45

2.2 Akses eksternal: psql host ke localhost:22122

Dari luar docker, client menyambung lewat port mapping host localhost:22122. Identitas server dan alamat IP internalnya dibaca langsung oleh psql (transkrip lengkap di bukti/07):

```
$ psql --version
psql (PostgreSQL) 18.3 (Debian 18.3-1+b1)

== identitas server (dilihat dari luar) ==
PostgreSQL 15.19 on x86_64-pc-linux-musl, compiled by gcc (Alpine 15.2.0)

 ip_dalam_docker | port_dalam_docker
-----------------+------------------
 172.24.0.2      |              5432

== \conninfo sebagai bi_dev ==
 Database              | uts_mbd
 Client User           | bi_dev
 Host Address          | 127.0.0.1
 Server Port           | 22122
 Password Used         | true
```

Ketiga role (backend_dev, bi_dev, data_engineer) diuji dari jalur eksternal yang sama; hasil uji hak aksesnya dirinci di Bab 4. Fakta bahwa client 127.0.0.1:22122 sampai ke server 172.24.0.2:5432 di dalam docker adalah bukti port mapping bekerja dari luar.

2.3 Akses eksternal lewat DBeaver

Koneksi tersimpan di Database Navigator DBeaver dengan JDBC URL jdbc:postgresql://localhost:22122/uts_mbd dan user backend_dev. Port terbuka di host dibuktikan netstat (transkrip lengkap di bukti/08 dan bukti/10, setelah hardening mengikat ke 127.0.0.1):

```
=== netstat -ano (baris memuat 22122 dan 44122, setelah hardening) ===
TCP    127.0.0.1:22122    0.0.0.0:0    LISTENING    15676
TCP    127.0.0.1:44122    0.0.0.0:0    LISTENING    15676
(PID 15676 = Docker Desktop backend; 127.0.0.1 = hanya komputer lokal)

Langkah membuka di DBeaver:
1. klik ganda koneksi 'uts_mbd localhost:22122'
2. password backend_dev_122 diminta sekali lalu tersimpan
3. jalankan db/uji/dbeaver_eksternal.sql (Alt+X) atau @include
```

Catatan jujur: sesi DBeaver sempat ditutup manual sebelum tangkapan layar final diambil, sehingga tangkapan GUI DBeaver tidak ikut dilampirkan. Jalur eksternal sendiri tetap terbukti utuh lewat psql host (2.2) untuk ketiga role. Membuka ulang di DBeaver cukup 30 detik dengan langkah tiga baris di atas karena koneksinya sudah tersimpan.

2.4 Dokumentasi Docker Desktop

Docker Desktop 4.71.0 (Engine 29.4.1) dipakai sebagai runtime. Status kedua container terdokumentasi lewat docker ps pada transkrip 1.4 (postgres_ariel Up dan healthy, pgadmin_ariel Up), dan seluruh aktivitas compose (up, logs, exec, ps) tercatat di folder bukti/.

2.5 Fitur pembeda pada bab ini

- **FITUR PEMBEDA: Register server dua arah dengan dua identitas berbeda.** pgAdmin memakai nama container postgres_ariel:5432 (internal), DBeaver dan psql memakai localhost:22122 (eksternal); perbedaan dua jalur ini dibuktikan dengan transkrip, bukan klaim.
- **FITUR PEMBEDA: pgAdmin mode desktop (SERVER_MODE False).** Sesi pgAdmin hanya sebatas browser lokal pengerja, sesuai sifat tugas individu.

3. Skema SALAM, Tabel, dan Constraint yang Terbukti Bekerja (bobot 20 poin)

Soal meminta schema SALAM dan tabel mahasiswas dengan PRIMARY KEY, UNIQUE, dan CHECK yang terbukti bekerja. Di sini ketiganya dibuat bersama fitur tambahan (tabel nilai berelasi, trigger audit, view rekap) supaya pekerjaan ini punya ciri khas tersendiri. Skrip lengkap ada di db/init/01_schema.sql; di bawah ini intinya.

3.1 Definisi inti (dari 01_schema.sql)

```
CREATE SCHEMA SALAM;   -- tersimpan 'salam' di katalog (lowercase)

CREATE TABLE salam.mahasiswas (
    id             BIGINT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
    nim            VARCHAR(15)  NOT NULL,
    nama           VARCHAR(100) NOT NULL,
    email          VARCHAR(120) NOT NULL,
    jenis_kelamin  CHAR(1)      NOT NULL,
    angkatan       SMALLINT     NOT NULL,
    ipk            NUMERIC(3,2) NOT NULL DEFAULT 0.00,
    semester       SMALLINT     NOT NULL,
    status         VARCHAR(12)  NOT NULL DEFAULT 'aktif',
    created_at     TIMESTAMPTZ  NOT NULL DEFAULT now(),
    CONSTRAINT uq_mahasiswas_nim    UNIQUE (nim),
    CONSTRAINT uq_mahasiswas_email  UNIQUE (email),
    CONSTRAINT jk_valid        CHECK (jenis_kelamin IN ('L','P')),
    CONSTRAINT angkatan_range  CHECK (angkatan BETWEEN 2018 AND 2026),
    CONSTRAINT ipk_range       CHECK (ipk >= 0.00 AND ipk <= 4.00),
    CONSTRAINT semester_range  CHECK (semester BETWEEN 1 AND 14),
    CONSTRAINT status_valid    CHECK (status IN
                                  ('aktif','cuti','lulus','keluar'))
);
```

Struktur hasilnya dibaca dari katalog (bukti/02):

```
$ docker exec postgres_ariel psql -U postgres -d uts_mbd -c "\dt salam.*"
 Schema |      Name      | Type |     Owner
--------+----------------+------+---------------
 salam  | mahasiswas     | table | data_engineer
 salam  | nilais         | table | data_engineer
 salam  | riwayat_status | table | data_engineer

$ docker exec postgres_ariel psql -U postgres -d uts_mbd -c "\dv salam.*"
 salam  | v_rekap_mahasiswa | view | data_engineer
```

3.2 Uji constraint: pelanggaran sengaja dimasukkan, semua ditolak

Skrip db/uji/uji_constraint.sql menjalankan sepuluh uji (A sampai J). Setiap pelanggaran memang harus mengeluarkan ERROR; error itulah buktinya. Ringkasan hasil (transkrip lengkap di bukti/03):

Uji	Yang dilanggar	Respons PostgreSQL
A	PRIMARY KEY: INSERT id yang sudah ada	ERROR: duplicate key value violates unique constraint "mahasiswas_pkey"
B	UNIQUE nim: NIM dipakai ulang	ERROR: duplicate key ... violates

		unique constraint "uq_mahasiswas_nim"
C	UNIQUE email: email dipakai ulang	ERROR: duplicate key ... violates unique constraint "uq_mahasiswas_email"
D	CHECK ipk 4.50 (maks 4.00)	ERROR: new row ... violates check constraint "ipk_range"
E	CHECK semester 15 (maks 14)	ERROR: ... violates check constraint "semester_range"
F	CHECK status di luar daftar	ERROR: ... violates check constraint "status_valid"
G	CHECK nilai_huruf Z di tabel nilais	ERROR: ... violates check constraint "nilai_huruf_valid"
H	FK ON DELETE CASCADE	DELETE 1 lalu nilai milik mahasiswa itu tersisa 0 (ikut terhapus)
I	Trigger audit perubahan status	UPDATE 1 lalu tabel riwayat_status terisi otomatis: aktif -> lulus
J	Transaksi SAVEPOINT	insert pelanggar dibatalkan sebagian, dua insert sah tetap commit

Contoh transkrip uji H dan I (FK CASCADE dan trigger audit):

```
=== H. FK ON DELETE CASCADE ===
Sebelum delete: jumlah_nilai milik NIM 1247050115 = 1
DELETE 1
Sesudah delete: sisa_nilai = 0

=== I. TRIGGER audit: ubah status aktif -> lulus ===
UPDATE 1
 id | mahasiswa_id | status_lama | status_baru |     diubah_pada
----+--------------+-------------+-------------+---------------------
  2 |            3 | aktif       | lulus       | 2026-11-01 09:33:23
```

Contoh transkrip uji J (transaksi parsial, SATU baris dibatalkan tanpa membatalkan sisanya):

```
BEGIN
INSERT 0 1                        -- 9999990101 sah
SAVEPOINT
ERROR: ... violates check constraint "ipk_range"   -- 9999990102 ipk 9.99
ROLLBACK                          -- hanya ke savepoint
INSERT 0 1                        -- 9999990103 sah
COMMIT
 nim        | nama
 9999990101 | Transaksi Satu     -- selamat
 9999990103 | Transaksi Dua      -- selamat, si pelanggar tidak masuk
```

3.3 Fitur pembeda pada skema

- **FITUR PEMBEDA: Tabel nilais berelasi ke mahasiswas dengan FK ON DELETE CASCADE, UNIQUE gabungan (mahasiswa_id, kode_matkul, semester_diambil), dan CHECK enum nilai huruf A/AB/B/BC/C/D/E.** Satu mahasiswa tidak bisa menyimpan nilai ganda untuk mata kuliah yang sama di semester yang sama, dijaga database bukan aplikasi.

- **FITUR PEMBEDA: Trigger audit trg_riwayat_status + tabel riwayat_status.** Setiap perubahan status mahasiswa dicatat otomatis beserta status lama, status baru, dan waktunya (contoh activity khas yang dibuktikan di uji I).
- **FITUR PEMBEDA: View v_rekap_mahasiswa (LEFT JOIN + agregasi COUNT/SUM/ROUND).** Menjumlahkan mata kuliah, total SKS, dan rata-rata nilai per mahasiswa; dipakai bi_dev sebagai contoh view yang hanya bisa dibaca.
- **FITUR PEMBEDA: COMMENT ON pada schema, tabel, kolom, dan view.** Dokumentasi ikut tersimpan di katalog PostgreSQL, terbaca lewat pg_description oleh siapa pun yang membuka database.
- **FITUR PEMBEDA: Data awal memuat NIM sendiri (1247050122) plus tiga mahasiswa lain.** Menandakan pengerjaan individual, terlihat di Screenshot 5 dan 7.
- **FITUR PEMBEDA: Demonstrasi transaksi SAVEPOINT dan ROLLBACK TO SAVEPOINT.** Dua insert sah selamat, satu insert pelanggar CHECK dibatalkan sebagian (uji J).

4. Tiga User SQL-script: backend_dev, bi_dev, data_engineer (bobot 20 poin)

Soal meminta tiga user lewat SQL-script: backend_dev dengan CRUD semua tabel, bi_dev yang hanya boleh SELECT semua tabel dan view, serta data_engineer yang boleh CREATE, MODIFY (ALTER), dan DROP semua objek plus CRUD semua tabel. Skripnya db/init/02_roles.sql, dijalankan otomatis saat boot pertama.

4.1 Definisi inti (dari 02_roles.sql)

```
CREATE ROLE backend_dev   LOGIN PASSWORD 'backend_dev_122';
CREATE ROLE bi_dev        LOGIN PASSWORD 'bi_dev_122';
CREATE ROLE data_engineer LOGIN PASSWORD 'data_engineer_122';

-- Di PostgreSQL hanya OWNER yang boleh ALTER/DROP objek,
-- maka kepemilikan dipindahkan ke data_engineer:
ALTER SCHEMA salam OWNER TO data_engineer;
ALTER TABLE  salam.mahasiswas     OWNER TO data_engineer;
ALTER TABLE  salam.nilais         OWNER TO data_engineer;
ALTER TABLE  salam.riwayat_status OWNER TO data_engineer;
ALTER VIEW   salam.v_rekap_mahasiswa OWNER TO data_engineer;
GRANT CREATE ON DATABASE uts_mbd TO data_engineer;

GRANT USAGE ON SCHEMA salam TO backend_dev;
GRANT SELECT, INSERT, UPDATE, DELETE ON ALL TABLES IN SCHEMA salam
    TO backend_dev;
GRANT USAGE, SELECT ON ALL SEQUENCES IN SCHEMA salam TO backend_dev;

GRANT USAGE ON SCHEMA salam TO bi_dev;
GRANT SELECT ON ALL TABLES IN SCHEMA salam TO bi_dev;

-- Objek NANTI yang dibuat data_engineer tetap mewarisi pembagian hak:
ALTER DEFAULT PRIVILEGES FOR ROLE data_engineer IN SCHEMA salam
    GRANT SELECT, INSERT, UPDATE, DELETE ON TABLES TO backend_dev;
ALTER DEFAULT PRIVILEGES FOR ROLE data_engineer IN SCHEMA salam
    GRANT SELECT ON TABLES TO bi_dev;
```

4.2 Hak per role dan buktinya

Ketiganya diuji dari jalur eksternal localhost:22122, jadi tabel ini sekaligus bukti tambahan akses luar docker. Baris GAGAL memang harus gagal; permission denied itulah buktinya.

Role	Boleh	Ditolak (terbukti)	Transkrip
backend_dev	SELECT, INSERT, UPDATE, DELETE semua tabel; SELECT view	CREATE TABLE: permission denied for schema salam	bukti/04
bi_dev	SELECT semua tabel dan view	INSERT, UPDATE, DELETE, CREATE TABLE semuanya permission denied	bukti/05
data_engineer	CREATE SCHEMA/TABLE, ALTER (tambah kolom, rename), DROP, dan CRUD semua tabel sebagai owner	tidak ada yang ditolak untuk objek proyek (memang pemiliknya)	bukti/06
data_engineer + backend_dev (objek baru)	tabel baru buatan data_engineer otomatis bisa di-CRUD backend_dev	-	bukti/09

Cuplikan bukti backend_dev (CRUD penuh, CREATE ditolak):

```
$ PGPASSWORD=backend_dev_122 psql -h localhost -p 22122 -U backend_dev -d uts_mbd

=== 3. INSERT (Create) ===   INSERT 0 1   -> id 10 | 1247050199 | Uji Backend Dev
=== 4. UPDATE ===            UPDATE 1     -> ipk menjadi 3.42
=== 6. DELETE ===            DELETE 1     -> sisa nilai terkait = 0 (CASCADE)

=== 7. [GAGAL - diharapkan] CREATE TABLE ditolak ===
ERROR:  permission denied for schema salam
```

Cuplikan bukti bi_dev (hanya baca):

```
=== 1. SELECT tabel mahasiswas (BOLEH) ===
  1 | 1247050122 | M. Ariel Davinsyah Putra | 3.45 | 5 | aktif

=== 4-7. [GAGAL - diharapkan] ===
ERROR:  permission denied for table mahasiswas   (INSERT, UPDATE, DELETE)
ERROR:  permission denied for schema salam       (CREATE TABLE)
```

Cuplikan bukti data_engineer (DDL penuh):

```
=== 1. CREATE schema + tabel baru ===   CREATE SCHEMA / CREATE TABLE
=== 3. ALTER TABLE: tambah kolom ===    ALTER TABLE (2x)
=== 5. ALTER objek proyek: rename kolom mahasiswas lalu dikembalikan ===
ALTER TABLE / ALTER TABLE
=== 7. DROP tabel + schema buatan ===   DROP TABLE / DROP SCHEMA
=== 8. View v_rekap tetap terbaca, objek proyek utuh ===
 1247050122 | M. Ariel Davinsyah Putra | 3 | 10
```

Cuplikan bukti default privileges (bukti/09): tabel baru dibuat data_engineer, langsung bisa diubah backend_dev tanpa grant manual:

```
data_engineer> CREATE TABLE salam.tes_default_priv (...); INSERT ...;
CREATE TABLE / INSERT 0 1
backend_dev>   SELECT ... ; UPDATE salam.tes_default_priv SET isi = ...;
  1 | dibuat data_engineer
UPDATE 1
data_engineer> DROP TABLE salam.tes_default_priv;
DROP TABLE
```

Daftar role yang tercipta terbaca dari katalog (bukti/02):

```
$ psql -U postgres -d uts_mbd -c "\du"
   Role name   |                         Attributes
---------------+------------------------------------------------------------
 backend_dev   |
 bi_dev        |
```

```
 data_engineer |
 postgres      | Superuser, Create role, Create DB, Replication
```

4.3 Fitur pembeda pada pengaturan role

- **FITUR PEMBEDA: Kunci desain: kepemilikan objek dipindahkan ke data_engineer.** Di PostgreSQL GRANT saja tidak cukup untuk ALTER/DROP; hanya owner yang boleh. Karena itu 02_roles.sql memindahkan ownership, bukan memberi hak superuser (ketiga role tetap tanpa atribut khusus).
- **FITUR PEMBEDA: ALTER DEFAULT PRIVILEGES untuk objek masa depan.** Tabel yang nanti dibuat data_engineer otomatis tetap bisa di-CRUD backend_dev dan dibaca bi_dev; pembagian hak tidak bocor saat objek baru muncul (dibuktikan nyata di bukti/09).
- **FITUR PEMBEDA: Urutan eksekusi role setelah skema.** 02_roles.sql berjalan otomatis setelah 01_schema.sql pada boot pertama (terlihat di log Bab 1), sehingga clone-an repo langsung siap diuji tanpa langkah manual.

5. Push ke Github (bobot 10 poin)

Seluruh berkas proyek (docker-compose.yml, .env, .env.example, skrip SQL init dan uji, transkrip bukti, README, serta laporan ini) dipush ke repositori publik berikut. Berkas soal tidak ikut dipublikasikan karena materi milik dosen; inti ketentuannya sudah tersalin di laporan ini dan README:

https://github.com/kemalzaki/uts-manajemen-basis-data

5.1 Isi repositori

Berkas / folder	Isi
docker-compose.yml	definisi postgres_ariel dan pgadmin_ariel, tanpa kredensial hardcode
.env / .env.example	kredensial dan port; sengaja di-commit demi penilaian (catatan di README)
db/init/	01_schema.sql (skema SALAM) dan 02_roles.sql (tiga role), auto-init
db/uji/	lima skrip pengujian: constraint, tiga role, dbeaver eksternal
bukti/	sepuluh transkrip output asli pengujian (termasuk bukti hardening port)
laporan/	laporan ini dalam format DOCX dan PDF
README.md	dokumentasi lengkap: cara menjalankan, tabel hak role, fitur pembeda

5.2 Cara memverifikasi ulang (reproduce)

```
git clone https://github.com/kemalzaki/uts-manajemen-basis-data.git
cd uts-manajemen-basis-data
cp .env.example .env          # atau pakai .env yang sudah ada
docker compose up -d          # tunggu postgres_ariel healthy
docker ps --filter name=ariel

# akses internal:  browser -> http://localhost:44122 (login .env)
#                   register server: host postgres_ariel, port 5432
# akses eksternal: psql/DBeaver -> localhost:22122, user backend_dev dkk.

# uji ulang seluruh bukti:
docker exec -i postgres_ariel psql -U postgres -d uts_mbd -f - < db/uji/uji_constraint.sql
PGPASSWORD=backend_dev_122 psql -h localhost -p 22122 -U backend_dev -d uts_mbd -f
db/uji/uji_backend_dev.sql
PGPASSWORD=bi_dev_122        psql -h localhost -p 22122 -U bi_dev        -d uts_mbd -f
db/uji/uji_bi_dev.sql
PGPASSWORD=data_engineer_122 psql -h localhost -p 22122 -U data_engineer -d uts_mbd -f
db/uji/uji_data_engineer.sql
```

Karena data hidup di volume pgdata_ariel, pengujian ulang pada volume yang sudah berisi data lama tidak menjalankan auto-init; untuk mulai dari nol gunakan docker compose down -v lalu up -d lagi (tertulis juga di README).

6. Penutup: Pemetaan Bukti ke Kriteria Penilaian

Tabel berikut merangkum di mana setiap kriteria soal dibuktikan:

Kriteria soal	Bobot	Dibuktikan di
Container postgres_NAMA,	30	Bab 1: transkrip up + ps + simulasi

PostgreSQL 9-15, user postgres password ifunggul via .env, port 22XXX		tanpa .env + log init; versi 15.19 terbaca dari luar (Bab 2.2)
Container pgadmin_NAMA port 44XXX + bukti akses internal (PGAdmin4) dan eksternal (DBeaver)	20	Bab 2: Screenshot 1-7 (internal postgres_ariel:5432), bukti/07 psql tiga role + bukti/08 DBeaver localhost:22122
Schema SALAM + tabel mahasiswas: PK, UNIQUE, CHECK terbukti bekerja	20	Bab 3: sepuluh uji A-J ditolak sesuai aturan (bukti/03), struktur katalog (bukti/02)
SQL-script users backend_dev / bi_dev / data_engineer terbukti	20	Bab 4: CRUD backend_dev, read-only bi_dev, DDL+CRUD data_engineer, default privileges (bukti/04-06, 09)
Push docker compose + scripts ke Github	10	Bab 5: repositori publik beserta seluruh skrip dan bukti

Catatan jujur terakhir: satu-satunya bukti yang tidak dilampirkan dalam bentuk tangkapan layar adalah GUI DBeaver yang sedang terhubung (penjelasan di 2.3). Koneksinya tersimpan dan langkah membukanya tiga baris; jalur eksternal yang diwakilinya tetap terbukti penuh lewat psql untuk ketiga role. Semua tangkapan layar lain diambil langsung dari mesin pengerjaan dan dapat direproduksi dengan langkah pada 5.2.